JavaScript Study Notes

Practice file: app.js — Loops, Arrays & prompt()

1. Basic for Loop

A for loop repeats a block of code a set number of times. It has three parts separated by semicolons: initialization, condition, and increment/decrement.

```
for (var i = 0; i < 10; i++) {
    console.log("hello");
}
```

i = 0 runs once at the start. **i < 10** is checked before every loop; the loop stops as soon as it's false. **i++** runs after each pass. This prints "hello" 10 times (i = 0 through 9).

2. Arrays

An array stores a list of values in one variable. Items are accessed by index, starting at 0.

```
var city = ["karachi", "lahore", "islamabad", "multan", "pindi"];
```

city[0] is "karachi", **city[4]** is "pindi". **city.length** gives the number of items (5 here) — useful as the loop limit so the loop always matches the array's actual size.

Looping through an array

```
var city = ["karachi","lahore","islamabad","multan","pindi","hyd"];
for (var i = 0; i < city.length; i++) {
    console.log(city[i]);
}
```

Using **city.length** instead of a hardcoded number (like 6) means the loop still works correctly if items are added or removed from the array later.

3. prompt() — Getting User Input

prompt() opens a small popup box asking the user to type something. Whatever they type is returned as a **string** and can be stored in a variable.

```
var userInput = prompt("Enter your city name: ");
```

> Good to know: prompt() always returns text (a string), even if the user types numbers. That's why the multiplication table examples below use the **+** trick to convert the input into a real number.

4. Searching an Array — the "flag" Pattern

A common pattern for checking if a value exists in an array: set a boolean flag to false, loop through the array, and switch it to true if a match is found.

```
var flag = false;
for (var i = 0; i < city.length; i++) {
    if (city[i] === userInput) {
        console.log(userInput + " city found");
        flag = true;
        break;
    }
}
if (!flag) {
    console.log(userInput + " city not found");
}
```

Breaking this down:

- **flag = false** — starting assumption: "not found yet."
- **===** (strict equality) compares both value and type — safer than **==**.
- **break** exits the loop immediately once a match is found, so it doesn't keep checking the rest of the array unnecessarily.
- **!flag** means "if flag is false" — i.e. the loop finished without finding a match.

5. Multiplication Table (Nested Logic with prompt)

Two numbers are collected from the user, then a loop prints a table by multiplying the base number by every value from 1 up to the chosen length.

```
var num = +prompt("enter you number: ");
var len = +prompt("enter len: ");
for (var i = 1; i <= len; i++) {
    console.log(num + " x " + i + "=" + num * i);
}
```

The leading **+** before prompt() converts the returned string into a number. Without it, **num * i** could behave unexpectedly since prompt() input starts out as text.

Counting down (reverse loop)

```
var num = +prompt("enter you number: ");
for (var i = 10; i >= 1; i--) {
    console.log(num + " x " + i + "=" + num * i);
}
```

Same idea, but the loop starts at 10 and counts down to 1 using **i--**, printing the table in reverse order.

6. Key Takeaways

- Use **array.length** in loop conditions instead of a fixed number.
- prompt() always returns a string — use **+prompt(...)** to get a number.
- Use **===** for comparisons, not **==**.
- The flag + break pattern is a standard way to search an array and stop early once found.
- A for loop's three parts control start point, stop condition, and step direction (++ or --).

> Practice tip: try rewriting the search-and-flag example using **indexOf()** or **includes()** once you reach array methods — same logic, less code.